

Northern University of Business and Technology Khulna

[Project Report]

Project Title:

Ping Pong Game

Submitted by:

Name: Hridoy Laskar

ID: 11240321738

Course Name: Software Development I

Course Code: CSE 2216

Section: 4B

Submitted to:

Vashkar Kar

Lecturer,

Department of Computer Science and Engineering,

Northern University of Business and Technology Khulna

Date of Submission: 8-10-2026

Table of Contents

1. Abstract.....	3
2. Introduction.....	3
3. Methodology	4
4. Implementation	6
5. Results & Analysis.....	9
6. Discussion	11
7. Conclusion	12

1. Abstract

This project focuses on developing an interactive, 2D arcade-style application called the Ping Pong Game using the Java programming language. The main objective addressed in this project is creating a real-time GUI application in which the user controls a paddle to return a moving ball against a computer-controlled opponent, while the system tracks the score and declares a winner. The system uses custom vector drawing via Java Swing and AWT to render the table, two paddles, the ball, and the live score display.

The methods implemented include continuous timer-based rendering (`javax.swing.Timer` with a 14 ms interval), bounding-box collision detection (`Rectangle.intersects`), keyboard event handling (`KeyListener` with a set of currently pressed keys), a simple AI opponent that tracks the ball, a scoring system with a win condition, and pause/restart controls. The results show that the application reliably updates paddle and ball positions, handles wall and paddle collisions, tracks both players' scores, and manages the running, paused, and game-over states smoothly. This project demonstrates the practical application of event-driven programming and real-time graphics handling in Java.

2. Introduction

Background

Java is an object-oriented programming language widely recognized for its platform independence, robust memory management, and rich graphical library support. Developing 2D interactive applications provides an effective way to master event-driven programming and graphical user interface (GUI) development. The Swing and AWT frameworks allow developers to draw custom 2D graphics, process real-time keyboard input, and run animation loops using built-in timers.

Ping Pong (Pong) is one of the earliest and simplest arcade games. Its small set of rules, two paddles, a ball, and a score, makes it an ideal project for learning how game loops, collision detection, and game state management work together.

Problem Statement

Understanding event-driven GUI architecture, real-time animation loops, and collision physics can be challenging when approached solely through theoretical concepts. Static learning often misses the complexity of frame updates, continuous keyboard input, state tracking, and opponent behavior. Therefore, constructing an interactive Ping Pong game

addresses these educational challenges by applying programming concepts to solve real-time rendering, input-handling, and game-logic problems.

Objectives

The objectives of this project are:

- To design and implement an interactive 2D Ping Pong Game using Java Swing and AWT.
- To implement a game loop using `javax.swing.Timer` for continuous movement and rendering.
- To implement Player 1 paddle controls (W / S keys) using `KeyListener`, with boundary constraints so the paddle stays inside the play area.
- To apply bounding-box collision detection (`java.awt.Rectangle`) for ball-paddle collisions, and wall collision for the top and bottom edges.
- To implement real-time score tracking for both players and a win condition when a player reaches the score limit.
- To implement a basic AI opponent that controls the second paddle.
- To add pause and restart controls, and to apply exception handling for invalid states and edge cases.

Scope

The scope of this project includes a complete 2D desktop application built with Java Swing and AWT (`JFrame`, `JPanel`, `Graphics2D`). The application features a fixed 800×500 play area, two paddles, a ball with angle-adjusting bounces, a live score display, a pause screen, and a game-over screen announcing the winner. The project operates in a local desktop environment and does not include network multiplayer functionality, audio, or data persistence.

3. Methodology

Approach

The project follows an event-driven approach over a 10-week plan, moving from planning and design to implementation, polishing, testing, and documentation. A `javax.swing.Timer` triggers an action event every 14 milliseconds. During each timer tick, if the game is neither paused nor over, the system updates the player's paddle position, updates the AI paddle

position, moves the ball, checks collisions and scoring, and then requests a repaint of the panel.

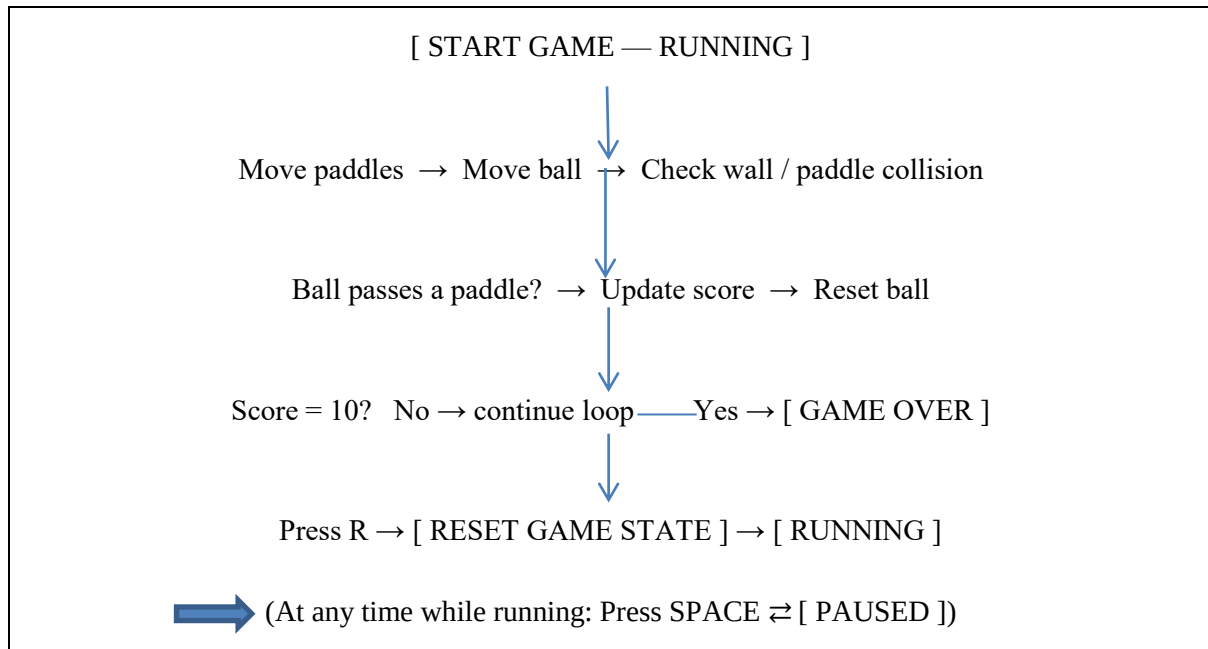
Algorithm

Algorithm for the Ping Pong Game Loop and State Logic:

1. Start the application and create the main window containing an 800×500 pixel game panel.
2. Set default parameters: both paddles centered vertically (Player 1 at $x = 30$, Player 2 at the right side), ball at the center with speed (5, 4), both scores = 0, isPaused = false, gameOver = false.
3. Start the Timer with a 14 ms interval.
4. Intercept keyboard inputs (KeyListener) and keep the set of currently pressed keys:
 - W / S keys move the Player 1 paddle up / down.
 - SPACE toggles pause (only while the game is not over).
 - R restarts the game.
5. On each timer event, if the game is not paused and not over:
 - Move Player 1's paddle according to the pressed keys, and clamp it within the play area.
 - Move the AI paddle toward the ball's vertical position when the ball is travelling toward it, and clamp it within the play area.
 - Add the ball's velocity to its position.
6. If the ball touches the top or bottom edge, reverse its vertical velocity.
7. Create bounding boxes for the ball and both paddles. If the ball intersects a paddle (and is moving toward it), place the ball at the paddle's edge, reverse its horizontal velocity, and adjust the vertical velocity depending on whether it hit the upper, middle, or lower part of the paddle.
8. If the ball passes the left edge, increase Player 2's (AI) score; if it passes the right edge, increase Player 1's score. Then check the win condition and reset the ball to the center.
9. If a player's score reaches the winning score (10), set gameOver = true and display the result message with a restart prompt.

10. If any exception occurs during the timer event, restart the game to recover from the invalid state.
11. Repaint the graphics canvas (paintComponent).

Flowchart



Tools and Technologies

- **Programming Language:** Java (JDK 8 or higher)
- **GUI Libraries:** Java Swing (JFrame, JPanel, Timer) and AWT (Graphics2D, Rectangle, FontMetrics, Color, Font)
- **Development Environment (IDE):** NetBeans IDE
- **Hardware:** Laptop

Experimental Design

The system is verified by executing repeated gameplay runs across different input patterns and ball speeds. Tests verify paddle responsiveness, paddle boundary containment, ball–wall bouncing, ball–paddle collision accuracy at different speeds, correct score updates when the ball passes a paddle, the win condition at the score limit, AI paddle behavior, pause/resume behavior, and the restart function.

4. Implementation

System Architecture

The application is implemented in a single class, PingPongPaddles, which extends JPanel and is divided into three functional modules:

1. **Input & Control Module:** A KeyAdapter captures key presses and releases (W, S, SPACE, R) and stores the pressed keys in a HashSet, which allows smooth continuous paddle movement.
2. **Processing & Physics Module:** Driven by javax.swing.Timer, it handles paddle movement, AI behavior, ball movement, collision detection, scoring, and win/pause/game-over state updates.
3. **Graphics Output Module:** Uses Graphics2D inside paintComponent (with anti-aliasing enabled) to render the center line, paddles, ball, scores, and the pause and game-over messages.

Components

- **Timer Loop Controller:** Runs the update–repaint cycle every 14 ms and contains a try–catch block that restarts the game if an exception occurs.
- **Paddle Controller:** updatePaddlePositions() moves Player 1's paddle based on pressed keys, and clamp() keeps it inside the play area.
- **AI Opponent:** updateAIPaddle() moves the second paddle toward the ball's center only while the ball is moving toward it, with a small tolerance zone (± 10 px) to avoid jitter.
- **Ball & Collision Engine:** updateBallPosition() moves the ball, bounces it off the top and bottom edges, and uses Rectangle.intersects() for ball–paddle collisions.
- **Bounce Angle Logic:** bounceAngleAdjust() changes the ball's vertical speed by -1 , 0 , or $+1$ depending on whether it hits the upper, middle, or lower third of the paddle.
- **Scoring & Win Condition:** checkWinCondition() ends the game when either score reaches WINNING_SCORE, and resetBall() re-serves from the center.
- **State Controller:** The boolean flags isPaused and gameOver control which updates and which messages are active, and restartGame() resets scores, paddles, and ball.

Game Parameters

Parameter	Value
Play area size	800×500 pixels

Parameter	Value
Paddle size	15 × 100 pixels
Player paddle speed	6 pixels per tick
AI paddle speed	4 pixels per tick
Ball size	15 × 15 pixels
Initial ball speed	5 (horizontal), 4 (vertical)
Timer interval	14 ms
Winning score	10 points
Controls	W / S: move, SPACE: pause, R: restart

User Interface Design

The interface uses a dark background (RGB 20, 24, 30) with a dashed white center line. Player 1's paddle and score are shown in blue, the AI's paddle and score in red, and the ball in yellow, so each element is easy to identify. Scores are displayed in a large bold font at the top, in the left and right halves of the panel. When the game ends, the result text ("You Win!" or "Game Over! (AI Wins)") and a restart prompt are centered on the screen using FontMetrics; a "PAUSED" message is shown the same way when the game is paused.

Weekly Workflow

Week	Development Phase	Key Activities
Week 1	Project Planning & Concept Finalization	Select topic and objectives; define gameplay mechanics, controls, and scope; prepare initial project plan.
Week 2	Requirements Analysis & System Design	Gather functional requirements for paddle movement, scoring, and controls; design architecture and file layout; create wireframes for the canvas and score display.
Week 3	Environment Setup & Project Initialization	Install JDK and configure IDE; configure Swing environment; create project structure and main game window class.
Week 4	UI Layout & Game Window Development	Develop main game window and play area panel; create score display and status area; apply theme and basic styling.
Week 5	Paddle Movement & Controls	Implement Player 1 controls (W / S) using KeyListener; implement Player 2 / AI paddle control; add boundary constraints for paddles.
Week 6	Ball Physics & Collision Detection	Implement ball movement using Timer; add ball-paddle collision detection; add top and bottom wall collision.

Week	Development Phase	Key Activities
Week 7	Scoring System & Win Condition	Implement real-time score tracking; detect when the ball passes a paddle and update score; implement win condition at the score limit.
Week 8	AI Opponent & UX Polish	Implement basic AI paddle behavior; polish UI with a clean, modern look and visual feedback; optimize Timer interval for smooth gameplay.
Week 9	Error Handling & Testing	Add exception handling for invalid states and edge cases; test paddle and ball collision at different speeds; fix bugs and optimize performance.
Week 10	Final Testing, Documentation & Submission	Perform complete gameplay testing; prepare documentation and screenshots; finalize and submit the project.

5. Results & Analysis

Result

The application successfully executes a 2D Ping Pong environment with responsive keyboard controls and smooth ball motion. Testing confirmed that Player 1's paddle moves smoothly with the W and S keys and stays within the play area, and that the AI paddle follows the ball without leaving the play area. The ball bounces correctly off the top and bottom edges and off both paddles, with its vertical speed changing according to where it hits the paddle. When the ball passes a paddle, the opposing player's score increases and the ball is reset to the center. When a player reaches 10 points, the game stops and the result ("You Win!" or "Game Over! (AI Wins)") is displayed along with a restart prompt. The SPACE key pauses and resumes the game, and the R key restarts it.

Analysis

The results confirm the reliability of Swing's event-driven architecture when executing real-time 2D games. Using `javax.swing.Timer` keeps rendering synchronized with state updates without requiring manual thread management. Storing the currently pressed keys in a set allows smooth, continuous paddle movement instead of relying on the operating system's key-repeat behavior. Bounding-box detection via `Rectangle.intersects()`, combined with a check of the ball's direction and repositioning of the ball at the paddle's edge, prevents the ball from getting stuck inside a paddle. The AI paddle is intentionally slower than the ball

and only reacts when the ball is moving toward it, which makes it beatable and keeps the game fair. The try-catch block in the game loop ensures invalid states do not crash the application.

Output

The final outputs of the Ping Pong game are shown below. The interface displays the table with a dashed center line, the blue Player 1 paddle on the left, the red AI paddle on the right, the yellow ball, and the live scores at the top. The visual output is given below:

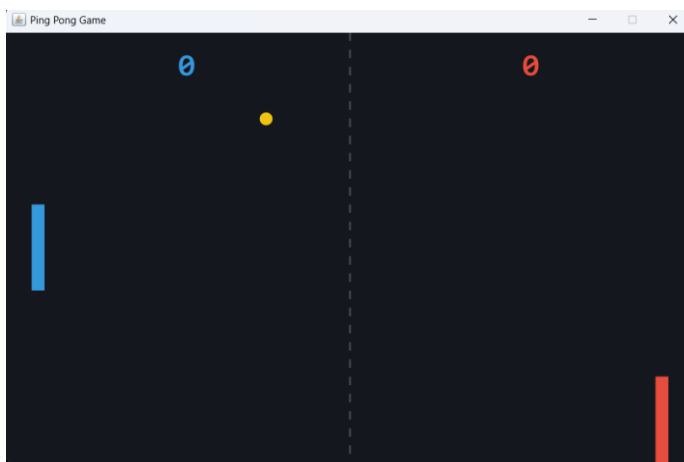


Figure 1: Active gameplay



Figure 2: Game over

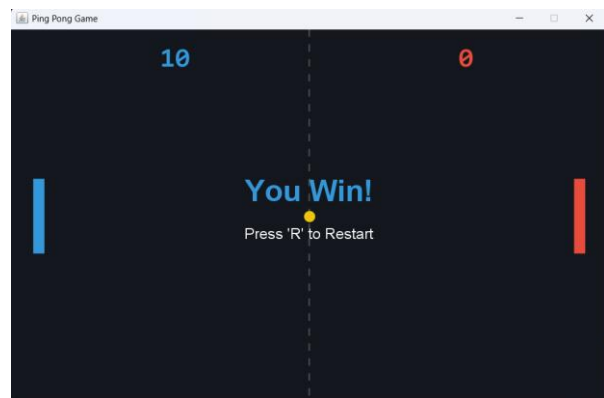


Figure 3: Win

6. Discussion

Interpretation

The implementation results demonstrate that Java Swing and AWT are fully capable of handling 2D game loops, vector graphics rendering, collision physics, and state management. Using simple boolean flags for the paused and game-over states keeps the code easy to follow and prevents updates from running at the wrong time. The system meets the project goals: responsive controls, accurate collision handling, real-time scoring with a win condition, a working AI opponent, and graceful handling of errors.

Key Technical Achievements

- **Smooth Input Handling:** Tracking pressed keys in a HashSet made paddle movement smooth and independent of key-repeat delays.
- **Collision Logic:** Using `Rectangle.intersects()` with a direction check and ball repositioning gave reliable ball–paddle collisions without the ball sticking inside a paddle.
- **Angle-Based Bouncing:** Adjusting the ball's vertical speed based on the paddle hit position adds control and variety to each rally.
- **AI Opponent:** The AI moves only when the ball approaches, uses a lower speed than the player, and has a tolerance zone, producing a challenging but beatable opponent.
- **State Management:** Pause, win condition, and restart are handled through simple flags and a single `restartGame()` method.

Challenges

- **Collision Accuracy:** Making the ball bounce off the paddle only once per hit required checking the ball's direction and repositioning it at the paddle's edge.
- **Smooth Gameplay:** Choosing a 14 ms Timer interval and suitable paddle and ball speeds was necessary for fluid gameplay.

- **AI Balance:** Making the AI challenging without being unbeatable required tuning its speed and reaction zone.
- **State Reset:** Resetting scores, paddles, and ball correctly on restart, and recovering from unexpected exceptions, required a single reliable reset method.

Limitations & Potential Enhancements

While the project meets all functional requirements, future iterations could expand the gameplay experience:

1. **Two-Player Mode:** Allowing a second human player to control the right paddle with the Up and Down arrow keys.
2. **Increasing Difficulty:** Gradually increasing the ball speed during a rally, adding difficulty levels for the AI, and limiting the maximum vertical speed.
3. **Sound Effects:** Adding audio feedback for paddle hits, scoring, and game over using the Java Sound API.
4. **Menu and High Scores:** Adding a start menu with mode selection and saving high scores to a local file.

7. Conclusion

Summary

The Java Ping Pong Game project successfully demonstrates how foundational Java concepts, such as event handling, Swing timers, vector graphics, collision detection, and state management, can be combined to build an engaging 2D game. By incrementally building the project from planning and design (Weeks 1–2) to window and UI development (Weeks 3–4), paddle controls (Week 5), ball physics and collision (Week 6), scoring and win condition (Week 7), AI opponent and UX polish (Week 8), and testing and documentation (Weeks 9–10), the project remained organized and met its objectives. The resulting application provides smooth controls, accurate collisions, a challenging AI opponent, clear visual feedback, and a simple restart mechanism.

Contributions

The project demonstrates how native Java libraries can build an interactive arcade game without external game engines. Key contributions include establishing a Swing timer game loop, implementing smooth keyboard-controlled paddle movement, building ball physics

with angle-adjusting collisions, developing a score and win-condition system, and creating a simple AI opponent. This project serves as an extensible foundation for learning Java GUI and 2D game development.